

- RAG Learning

4.2 Fundamentals of Data Structures

Computer Science — AQA A-Level (7517) — Paper 1: On-screen Programming

Topic 4.2 sits at the heart of Paper 1. Data structures provide the **shapes** in which data is stored so that algorithms can operate efficiently. Expect AQA to ask you to *describe* a data structure, *trace* its operations, *compare* alternative implementations, and *justify* which one to pick for a given scenario. You must know the difference between an **abstract data type** (the logical contract) and its possible **implementations** (how it is actually stored in memory).

4.2.1 Data Structures — General

A **data structure** is an organised collection of related data elements, together with a set of operations that may be performed on them. It describes *how* data is stored so that it can be accessed and modified efficiently.

Static vs Dynamic Data Structures

Aspect	Static	Dynamic
Size	Fixed at compile time; cannot grow or shrink.	Grows and shrinks at run time as items are added or removed.
Memory allocation	Allocated in one contiguous block on the stack or in the data segment.	Allocated on the <i>heap</i> , often in separately-linked nodes.
Typical example	Array of 100 integers.	Linked list, resizing (dynamic) array, tree.

Access time	Direct random access by index — $O(1)$.	Often sequential — $O(n)$ for a linked list.
Memory waste	Wastes space if only partially filled; overflows if the declared size is exceeded.	Uses exactly what it needs, but pays per-node overhead for pointers.

Abstract Data Type vs Implementation

An **abstract data type (ADT)** is a mathematical / logical model of a data structure, defined by the *operations* that may be performed on it and the *contract* those operations obey — independent of how they are programmed. An **implementation** is the concrete arrangement of memory (and code) that realises an ADT.

For instance, the *stack* ADT is specified by four operations (`push` , `pop` , `peek` , `isEmpty`) and the LIFO contract. That same ADT can be implemented using a fixed-size array plus an integer top-pointer, or using a singly-linked list of dynamically-allocated nodes. Client code that uses only the ADT operations will work with either implementation unchanged — this is the power of *abstraction*.

Exam Tip: If AQA asks "what is an abstract data type?" state: (1) a logical description of a data structure, (2) defined by its operations and their behaviour, (3) with the implementation hidden from the user. A two-mark answer needs at least two of these three points.

4.2.2 Single- and Multi-Dimensional Arrays

An **array** is a static data structure consisting of a fixed-size, ordered collection of elements *of the same data type*, stored in a single contiguous block of memory and accessed by an integer **index**.

Indexing and Bounds

- Most languages use **zero-based** indexing: a 5-element array has valid indices 0, 1, 2, 3, 4.
- AQA pseudocode is commonly **one-based** or allows the programmer to declare bounds explicitly: `DECLARE a : ARRAY[1..10] OF Integer`.
- Accessing an index outside the declared bounds raises an `IndexOutOfRangeException` exception (or produces undefined behaviour in C).
- Access is **O(1)**: the address of element `a[i]` is computed as `base + i × elementSize`.

Memory Layout of 2D Arrays — Row-Major vs Column-Major

Computer memory is one-dimensional, so a 2D array must be flattened. There are two widely-used conventions:

- **Row-major order** (C, C++, Java, Python NumPy default) — rows are stored one after another. For an `m × n` array, element `a[i][j]` sits at offset `(i × n + j) × elementSize`.
- **Column-major order** (Fortran, MATLAB, R) — columns are stored one after another. Element `a[i][j]` sits at offset `(j × m + i) × elementSize`.

```
Array:  a[0][0] a[0][1] a[0][2]
        a[1][0] a[1][1] a[1][2]
```

```
Row-major memory:  [a00] [a01] [a02] [a10] [a11] [a12]
```

```
Column-major memory: [a00] [a10] [a01] [a11] [a02] [a12]
```

The distinction matters for **cache performance**: iterating in the same order as storage minimises cache misses. A loop that traverses a row-major array row-by-row is much faster than column-by-column.

Higher-Dimensional Arrays

A **3D array** generalises naturally — e.g. `voxel[x][y][z]` for a cube of values. In row-major layout the address of `a[i][j][k]` in a `d1 × d2 × d3` array is `base + (i × d2 × d3 + j × d3 + k) × elementSize`. AQA occasionally asks students to compute such an offset.

```
// Declaring and using a 2D array in AQA pseudocode
DECLARE grid : ARRAY[1..3, 1..3] OF Integer
```

```
FOR r ← 1 TO 3
```

```
FOR c ← 1 TO 3
    grid[r, c] ← (r - 1) * 3 + c
ENDFOR
ENDFOR

OUTPUT grid[2, 3]    // prints 6
```

Exam Tip: If an AQA question gives you a 2D array and asks for the address of an element, always state the formula `base + (row × cols + col) × size` (row-major) or state your assumption explicitly. Examiners want to see the offset calculation, not just the final number.

4.2.3 Fields, Records and Files

A **record** is a *composite* (heterogeneous) data type consisting of a fixed collection of named **fields**, each of which may be of a different data type. A **file** is a named collection of records held on a backing store (disk, SSD, tape) and persisting beyond program execution.

Records

```
TYPE Student = RECORD
    studentID : Integer
    surname   : String
    forename  : String
    dob       : Date
    average   : Real
ENDRECORD

s : Student
s.studentID ← 14072
s.surname   ← "Lovelace"
```

Contrast with an array, which is *homogeneous* (every element has the same type). Records group logically-related information into a single entity that can be passed, returned and stored as a unit.

File Organisation

Organisation	How records are ordered	Access method	Typical use
Serial	Stored in the <i>order they arrive</i> , with no sort key.	Sequential — read from start until required record is found.	Transaction logs, append-only capture (e.g. till receipts during a day).
Sequential	Stored in <i>sorted order</i> of a key field.	Sequential — but can stop early once key exceeded; enables merge-style batch updates.	Master files in batch-processing systems (payroll, student records).
Random / Direct	Position computed from the key by a hash function.	Direct — seek straight to the computed address in $O(1)$ average time.	Online-transaction systems where instant look-up is required (bookings, stock).
Indexed-sequential	Sequential records plus a separate index that maps keys to block addresses.	Index look-up then sequential read within block.	Databases; ISAM; VSAM on mainframes.

Serial and sequential files differ only in ordering: both are read *in order*. Random files trade insert-order simplicity for near-constant look-up time at the cost of potential collisions (see hash tables below).

4.2.4 Abstract Data Types — Overview

AQA 7517 lists these ADTs by name: **queue**, **stack**, **graph**, **tree**, **hash table**, **dictionary** and **vector**. Each has a well-known contract and more than one standard implementation. The subsections below give the required depth for each.

ADT	Key operations	Contract (discipline)	Typical implementations
Queue	<code>enqueue</code> , <code>dequeue</code> , <code>peek</code> , <code>isEmpty</code> , <code>isFull</code>	First-In-First-Out (FIFO)	Circular array, linked list
Stack	<code>push</code> , <code>pop</code> , <code>peek</code> , <code>isEmpty</code> , <code>isFull</code>	Last-In-First-Out (LIFO)	Array + top pointer, linked list
Graph	<code>addVertex</code> , <code>addEdge</code> , <code>neighbours</code>	Set of vertices and edges	Adjacency matrix, adjacency list
Tree	<code>insert</code> , <code>delete</code> , <code>traverse</code>	Connected acyclic graph with a root	Linked nodes, array (heaps)
Hash table	<code>insert</code> , <code>lookup</code> , <code>remove</code>	Key-value map via hash function	Array + chaining, open addressing
Dictionary	<code>put</code> , <code>get</code> , <code>remove</code> , <code>contains</code>	Unique keys → values	Hash table, balanced BST
Vector	<code>get component</code> , <code>add</code> , <code>scalar multiply</code> , <code>dot product</code>	n-vector over $\mathbb{R}$: an index → real-number mapping	1-D array / list, dictionary (sparse), function

4.2.5 Queues

A **queue** is a First-In-First-Out (FIFO) abstract data type. Items are added at the *rear* (`enqueue`) and removed from the *front* (`dequeue`). The first element added is always the first one removed — mirroring a real-world queue at a supermarket till.

Standard Operations

- `enqueue(item)` — add `item` to the rear. Fails if `isFull()` .
- `dequeue()` — remove and return the front item. Fails if `isEmpty()` .

- `peek()` / `front()` — return the front item without removing it.
- `isEmpty()` — return `True` if the queue has no items.
- `isFull()` — return `True` if the queue has no space (bounded implementations only).

Array-Backed Linear Queue — the Problem

The naive implementation uses an array with two pointers (`front` and `rear`). Each `deQueue` increments the front pointer, so the queue *drifts* rightwards through the array and wastes the space at the left. Eventually `rear` hits the end even though the left of the array is empty.

Circular Queue (Ring Buffer)

The standard fix is to treat the array as *circular*: when a pointer passes the last index it wraps around to index 0 using the **modulo** operator.

```

CONSTANT SIZE = 5
DECLARE q      : ARRAY[0..SIZE-1] OF Integer
DECLARE front  : Integer ← 0
DECLARE rear   : Integer ← 0
DECLARE count  : Integer ← 0

PROCEDURE enqueue(item)
    IF count = SIZE THEN
        OUTPUT "Queue full"
    ELSE
        q[rear] ← item
        rear ← (rear + 1) MOD SIZE
        count ← count + 1
    ENDIF
ENDPROCEDURE

FUNCTION dequeue() RETURNS Integer
    IF count = 0 THEN
        OUTPUT "Queue empty" ; RETURN -1
    ELSE
        item ← q[front]
        front ← (front + 1) MOD SIZE
        count ← count - 1
        RETURN item
    
```

```
ENDIF
ENDFUNCTION
```

Worked Trace — Circular Queue of Size 5

Starting with `front = 0`, `rear = 0`, `count = 0`:

Step	Operation	front	rear	count	Array contents
1	enQueue(10)	0	1	1	[10, _, _, _, _]
2	enQueue(20)	0	2	2	[10, 20, _, _, _]
3	enQueue(30)	0	3	3	[10, 20, 30, _, _]
4	deQueue() → 10	1	3	2	[_ , 20, 30, _, _]
5	enQueue(40)	1	4	3	[_ , 20, 30, 40, _]
6	enQueue(50)	1	0	4	[_ , 20, 30, 40, 50]
7	enQueue(60)	1	1	5	[60, 20, 30, 40, 50]
8	enQueue(70)	Queue full — rejected			

Notice step 6: `rear` wraps from 4 to 0 via $(4 + 1) \text{ MOD } 5 = 0$. This is why modulo arithmetic is essential.

Linked-List Queue

An alternative implementation uses nodes on the heap, each with a `data` field and a `next` pointer. The queue object stores just two pointers: `head` (front) and `tail` (rear).

```
TYPE Node = RECORD
    data : Integer
    next : Pointer TO Node
ENDRECORD

head ← NULL
```

```
tail ← NULL

PROCEDURE enqueue(x)
    newNode ← NEW Node
    newNode.data ← x
    newNode.next ← NULL
    IF tail = NULL THEN
        head ← newNode
    ELSE
        tail.next ← newNode
    ENDIF
    tail ← newNode
ENDPROCEDURE
```

Priority Queue

A **priority queue** is a queue in which each element has an associated **priority**. `deQueue()` returns the element with the *highest* priority, not necessarily the one added earliest. Ties are usually broken by insertion order (FIFO).

Implementations include: a sorted linked list, an unsorted array scanned linearly for the highest priority, or a **binary heap** (the most efficient, giving $O(\log n)$ insert and remove). Priority queues underlie CPU scheduling, Dijkstra's shortest-path algorithm, and print-job queues.

Exam Tip: When a 3-mark question says "describe a queue", write all three of: (1) FIFO discipline, (2) `enqueue` at rear and `deQueue` at front, (3) a real-world example (print queue, keyboard buffer, OS process queue). One mark each.

4.2.6 Stacks

A **stack** is a Last-In-First-Out (LIFO) abstract data type. Items are *pushed* onto the top and *popped* from the top. Only the top item is ever accessible. Think of a pile of plates: you add and remove from the top; you never pull one out of the middle.

Standard Operations

- `push(item)` — place `item` on top of the stack. Fails if `isFull()` .
- `pop()` — remove and return the top item. Fails if `isEmpty()` .
- `peek()` / `top()` — return the top item without removing it.
- `isEmpty()` , `isFull()` — as for queues.

Array Implementation with a Top Pointer

```
CONSTANT SIZE = 100
DECLARE stack : ARRAY[0..SIZE-1] OF Integer
DECLARE top   : Integer ← -1      // -1 means empty

PROCEDURE push(item)
    IF top = SIZE - 1 THEN
        OUTPUT "Stack overflow"
    ELSE
        top ← top + 1
        stack[top] ← item
    ENDIF
ENDPROCEDURE

FUNCTION pop() RETURNS Integer
    IF top = -1 THEN
        OUTPUT "Stack underflow" ; RETURN -1
    ELSE
        item ← stack[top]
        top ← top - 1
        RETURN item
    ENDIF
ENDFUNCTION

FUNCTION peek() RETURNS Integer
    IF top = -1 THEN RETURN -1 ELSE RETURN stack[top] ENDIF
ENDFUNCTION
```

Worked Trace — Push / Pop

Starting with `top = -1` :

Step	Operation	top	Stack (bottom → top)	Return
1	push(7)	0	[7]	–
2	push(3)	1	[7, 3]	–
3	push(9)	2	[7, 3, 9]	–
4	peek()	2	[7, 3, 9]	9
5	pop()	1	[7, 3]	9
6	push(4)	2	[7, 3, 4]	–
7	pop()	1	[7, 3]	4
8	pop()	0	[7]	3

Array vs Linked-List Implementation

Aspect	Array + top pointer	Linked list
Memory	Pre-allocated, may waste space or overflow.	One node per item, but each node carries pointer overhead.
Size	Fixed maximum (static).	Grows and shrinks freely (dynamic).
Push / pop	$O(1)$ — direct index access.	$O(1)$ — manipulate head pointer.
Cache behaviour	Excellent — contiguous memory.	Poor — nodes scattered across heap.
Overflow	Possible if SIZE is exceeded.	Only if the entire heap is exhausted.

Applications of Stacks

- **Expression evaluation** — in particular Reverse Polish (postfix) notation.
- **Function call / recursion support** — each call pushes a stack frame; **RETURN** pops one.

- **Undo / redo** in editors — each edit pushes a reversible action; `undo` pops.
- **Back button** in web browsers — URLs pushed on navigation.
- **Depth-first search** on graphs / trees — explicit stack replaces recursion.
- **Syntax parsing** — matching brackets, compiler symbol tables.

Worked Example — Reverse Polish Evaluation

Evaluate the postfix expression `5 1 2 + 4 * + 3 -`. Read left to right: if it is a number, push it; if it is an operator, pop two operands, apply, push the result.

Token	Action	Stack after
5	push 5	[5]
1	push 1	[5, 1]
2	push 2	[5, 1, 2]
+	pop 2, pop 1, push 1+2=3	[5, 3]
4	push 4	[5, 3, 4]
*	pop 4, pop 3, push 3×4=12	[5, 12]
+	pop 12, pop 5, push 5+12=17	[17]
3	push 3	[17, 3]
-	pop 3, pop 17, push 17-3=14	[14]

At end-of-input, the single value remaining on the stack (`14`) is the result. Infix equivalent: `(5 + (1 + 2) × 4) - 3 = 14`.

Exam Tip: AQA's favourite stack question is a Reverse Polish trace. Present your answer as a three-column table (Token, Action, Stack after). Always show the stack contents explicitly after every token — the marks are awarded per row, not per final answer.

4.2.7 Graphs

A **graph** is a non-linear data structure consisting of a set of **vertices** (nodes) and a set of **edges** (connections) between pairs of vertices. Formally, $G = (V, E)$.

Graph Terminology

- **Directed graph (digraph)** — each edge has a direction, e.g. one-way streets; edge $(A, B) \neq$ edge (B, A) .
- **Undirected graph** — edges are unordered pairs; connection works both ways.
- **Weighted graph** — each edge carries a numerical weight (distance, cost, capacity).
- **Connected graph** — a path exists between every pair of vertices.
- **Cycle** — a path that starts and ends at the same vertex via distinct edges.
- **Degree** of a vertex — the number of edges incident to it (in-degree and out-degree for digraphs).
- **Tree** — a connected, acyclic, undirected graph. A tree on n vertices has exactly $n - 1$ edges.

Adjacency Matrix Representation

A $V \times V$ matrix where $M[i][j]$ is 1 (or the edge weight) if there is an edge from vertex i to vertex j , and 0 otherwise.

Undirected graph: A–B, A–C, B–D, C–D

	A	B	C	D
A	[0, 1, 1, 0]			
B	[1, 0, 0, 1]			
C	[1, 0, 0, 1]			
D	[0, 1, 1, 0]			

For undirected graphs the matrix is symmetric along the leading diagonal. Storage is $O(V^2)$ regardless of edge count.

Adjacency List Representation

For each vertex, store a list of the vertices it is connected to. Total storage is $O(V + E)$.

```
A → [B, C]
B → [A, D]
C → [A, D]
D → [B, C]
```

When to Choose Which

Aspect	Adjacency Matrix	Adjacency List
Space	$O(V^2)$ — wasteful for sparse graphs.	$O(V + E)$ — efficient for sparse graphs.
Is there an edge i - j ?	$O(1)$ — single array look-up.	$O(\text{degree})$ — scan the list of i .
List all neighbours of i	$O(V)$ — scan a whole row.	$O(\text{degree})$ — direct iteration.
Add edge	$O(1)$.	$O(1)$ at head of list.
Best use	Small or dense graphs; matrix maths required.	Large sparse graphs (social networks, road networks).

Exam Tip: When AQA asks "which representation would you use for a road network of a city?" say *adjacency list* because road networks are sparse (each junction has only a few roads leaving it, so E is close to V not V^2). For a completely-connected tournament graph, say *adjacency matrix*.

4.2.8 Trees

A **tree** is a connected, acyclic graph with a designated **root** vertex. Every non-root vertex has exactly one parent. A tree with n vertices has exactly $n - 1$ edges. Nodes with no

children are called **leaves**.

Tree Terminology

- **Root** — the unique topmost node; it has no parent.
- **Parent / child** — immediate predecessor / successor.
- **Sibling** — nodes sharing the same parent.
- **Ancestor / descendant** — any node on the path from root / to leaves.
- **Leaf (external node)** — has no children.
- **Depth** of a node — number of edges from the root to that node.
- **Height** of a tree — maximum depth of any leaf (height of single-node tree is 0).
- **Subtree** — a node together with all its descendants.

Binary Trees

A **binary tree** is a tree in which every node has *at most two* children, conventionally called **left** and **right**. Special cases:

- **Full** binary tree — every internal node has exactly two children.
- **Complete** binary tree — every level is filled left-to-right, with only the last level possibly short.
- **Balanced** — for every node, the heights of its two subtrees differ by no more than 1.

Binary Search Tree (BST)

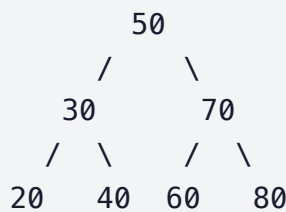
A **binary search tree (BST)** is a binary tree that maintains the *BST property*: for every node N , every key in N 's left subtree is strictly less than N 's key, and every key in its right subtree is strictly greater. This property enables $O(\log n)$ search, insert and delete on a balanced tree.

Building a BST — Worked Example

Insert the sequence **50, 30, 70, 20, 40, 60, 80** into an initially empty BST.

1. **50** becomes the root.

2. **30** < 50 → left child of 50.
3. **70** > 50 → right child of 50.
4. **20** < 50 → go left to 30; 20 < 30 → left child of 30.
5. **40** < 50 → go left to 30; 40 > 30 → right child of 30.
6. **60** > 50 → go right to 70; 60 < 70 → left child of 70.
7. **80** > 50 → go right to 70; 80 > 70 → right child of 70.



BST Insertion (Recursive Pseudocode)

```

TYPE TreeNode = RECORD
    key    : Integer
    left   : Pointer TO TreeNode
    right  : Pointer TO TreeNode
ENDRECORD

FUNCTION insert(node, key) RETURNS Pointer TO TreeNode
    IF node = NULL THEN
        newNode ← NEW TreeNode
        newNode.key ← key
        newNode.left ← NULL
        newNode.right ← NULL
        RETURN newNode
    ENDIF
    IF key < node.key THEN
        node.left ← insert(node.left, key)
    ELSE IF key > node.key THEN
        node.right ← insert(node.right, key)
    ENDIF
    RETURN node
ENDFUNCTION
  
```

Searching a BST

```

FUNCTION search(node, target) RETURNS Boolean
  IF node = NULL THEN RETURN False ENDIF
  IF node.key = target THEN RETURN True ENDIF
  IF target < node.key THEN
    RETURN search(node.left, target)
  ELSE
    RETURN search(node.right, target)
  ENDIF
ENDFUNCTION

```

Worst-case complexity is $O(h)$ where h is the tree's height. For a *balanced* tree $h = \lceil \log_2 n \rceil$, giving $O(\log n)$. A pathologically unbalanced tree (keys inserted in sorted order) degenerates to $h = n$, so search becomes $O(n)$.

Tree Traversals

A traversal visits every node exactly once in a defined order. The three canonical depth-first orders differ only in *where* the current node is visited relative to its subtrees.

Traversal	Order (for each node)	Result on example BST	Typical use
Pre-order	Node, Left, Right	50, 30, 20, 40, 70, 60, 80	Copy a tree; prefix notation.
In-order	Left, Node, Right	20, 30, 40, 50, 60, 70, 80	Visit BST in sorted order.
Post-order	Left, Right, Node	20, 40, 30, 60, 80, 70, 50	Delete / free a tree; postfix notation.

A **breadth-first** (level-order) traversal visits nodes level by level and requires a queue (not a stack). Result: 50, 30, 70, 20, 40, 60, 80.

```

PROCEDURE inOrder(node)
  IF node ≠ NULL THEN

```

```
        inOrder(node.left)
        OUTPUT node.key
        inOrder(node.right)
    ENDIF
ENDPROCEDURE

PROCEDURE preOrder(node)
    IF node ≠ NULL THEN
        OUTPUT node.key
        preOrder(node.left)
        preOrder(node.right)
    ENDIF
ENDPROCEDURE

PROCEDURE postOrder(node)
    IF node ≠ NULL THEN
        postOrder(node.left)
        postOrder(node.right)
        OUTPUT node.key
    ENDIF
ENDPROCEDURE
```

Exam Tip: For BST-insertion questions, always write out each insertion step comparing the new key against every node on the path. Examiners award a mark per correct comparison as well as for the final tree diagram.

General Rooted Trees

A general tree allows any number of children per node. It is often represented using *first-child / next-sibling* pointers so each node still needs only two pointer fields. File-system directories, XML/HTML document trees, and organisation charts are all general rooted trees.

4.2.9 Hash Tables

A **hash table** is a data structure that maps *keys* to *values* by passing the key through a **hash function** to obtain an array index. If the hash function spreads keys evenly, insert, look-up

and delete run in **O(1)** average time.

Properties of a Good Hash Function

- **Deterministic** — the same key always hashes to the same index.
- **Uniform distribution** — output values are spread evenly across the array's indices to minimise collisions.
- **Fast to compute** — hashing should add negligible overhead.
- **Use the full key** — every bit of the key should influence the output; otherwise similar keys collide.

```
// A simple hash function for an integer key
FUNCTION hash(key, tableSize) RETURNS Integer
    RETURN key MOD tableSize
ENDFUNCTION

// A simple hash function for a string key
FUNCTION hashString(s, tableSize) RETURNS Integer
    total ← 0
    FOR i ← 1 TO LENGTH(s)
        total ← (total * 31 + ASC(s[i])) MOD tableSize
    ENDFOR
    RETURN total
ENDFUNCTION
```

Collisions

A **collision** occurs when two distinct keys hash to the same index. The pigeonhole principle guarantees collisions are unavoidable in any table with more possible keys than slots.

Collision Resolution — Chaining

Each slot in the array holds a *linked list* (or other small container) of all keys that hashed there. Insertion adds to the front of the list; look-up scans the list.

```
index 0: → (key="tom", val=25)
index 1: → (key="ann", val=18) → (key="ian", val=42) // collision
index 2:
index 3: → (key="eve", val=31)
```

Collision Resolution — Open Addressing with Linear Probing

All entries live in the array itself. On collision, try the next slot, then the next, wrapping with modulo, until an empty slot is found.

```
FUNCTION insertLinear(key, value)
    i ← hash(key, SIZE)
    WHILE table[i] ≠ NULL AND table[i].key ≠ key DO
        i ← (i + 1) MOD SIZE
    ENDWHILE
    table[i] ← (key, value)
ENDFUNCTION
```

Linear probing is cache-friendly but suffers from **primary clustering**: long runs of occupied cells cause subsequent insertions to pile up. *Quadratic probing* (step by 1, 4, 9, 16, ...) and *double hashing* (step by a second hash of the key) reduce clustering.

Load Factor and Rehashing

The **load factor** λ of a hash table is the ratio n / size where n is the number of stored entries and size is the array length. As λ rises, the probability of collision rises, and average operation time degrades.

Rehashing keeps performance near $O(1)$: when λ exceeds a threshold (typically 0.7 for open addressing, 1.0 for chaining) the table is replaced with one roughly double the size, and every existing key is re-inserted using the new size. Amortised cost of insertion remains $O(1)$.

Exam Tip: For a 4-mark "explain how a hash table works" question, hit: (1) keys hashed via a hash function, (2) resulting index gives $O(1)$ access, (3) collisions resolved by chaining or open addressing, (4) load factor triggers rehashing when the table fills.

4.2.10 Dictionaries

A **dictionary** (also called an *associative array*, *map*, or *symbol table*) is an ADT storing **unique key → value** pairs with the operations `put(key, value)`, `get(key)`, `remove(key)` and `contains(key)`. Keys must support equality testing; values can be of any type.

Typical Implementations

- **Hash table** — average $O(1)$ put / get; no natural ordering of keys. This is how Python `dict`, Java `HashMap` and C# `Dictionary` work internally.
- **Balanced BST** (red-black or AVL) — guarantees $O(\log n)$ worst case and keeps keys in sorted order, supporting range queries. Java `TreeMap`, C++ `std::map`.
- **Sorted array + binary search** — $O(\log n)$ look-up but $O(n)$ insertion; viable when the dictionary is built once and then queried many times.

```
// AQA pseudocode dictionary
DECLARE ages : Dictionary
ages["Ada"]      ← 36
ages["Babbage"]  ← 79
ages["Turing"]   ← 41

OUTPUT ages["Turing"]           // 41
IF "Ada" IN ages THEN OUTPUT "known" ENDIF
REMOVE ages["Babbage"]
```

4.2.11 Vectors

In AQA 7517 a **vector** is a *mathematical* object, not a resizing array. An **n-vector over $\mathbb{R}$** is an ordered list of n real numbers, for example `[2.0, 3.0, -1.0]`. Equivalently it is a *function* from a finite set of n indices to the real numbers, or a *dictionary* mapping each index to its value. The set of all n -vectors over $\mathbb{R}$ is written $\mathbb{R}^n$.

Three Equivalent Representations

All three views describe exactly the same object — they differ only in how the index-to-value association is stored. Being able to move between them is examinable.

Representation	The vector [2.0, 3.0, -1.0]	Why you would choose it
List / 1-D array	<code>a = [2.0, 3.0, -1.0]</code> , accessed as <code>a[1]</code> , <code>a[2]</code> , <code>a[3]</code>	Compact and $O(1)$ to index; the natural choice when every component is stored.
Function	<code>a : {1, 2, 3} → ℝ</code> with <code>a(1) = 2.0</code> , <code>a(2) = 3.0</code> , <code>a(3) = -1.0</code>	The formal definition — a vector is a mapping from a finite index set to the reals. Useful when components are computed rather than stored.
Dictionary	<code>{1: 2.0, 2: 3.0, 3: -1.0}</code>	Best for <i>sparse</i> vectors: only the non-zero components are stored, and any missing key is taken to be 0.0.

// The same 3-vector, three ways (AQA pseudocode)

```
// 1. list / 1-D array
DECLARE a : ARRAY[1..3] OF Real
a[1] ← 2.0
a[2] ← 3.0
a[3] ← -1.0
```

```
// 2. function a : {1,2,3} → R
FUNCTION a(i)
    IF i = 1 THEN RETURN 2.0
    ELSE IF i = 2 THEN RETURN 3.0
    ELSE RETURN -1.0
    ENDIF
ENDFUNCTION
```

```
// 3. dictionary (sparse: absent key means 0.0)
DECLARE sparse : Dictionary
sparse[2] ← 7.0
```

```
sparse[900] ← 1.5
// this represents a 1000-vector whose other 998 components are 0.0
```

Notation

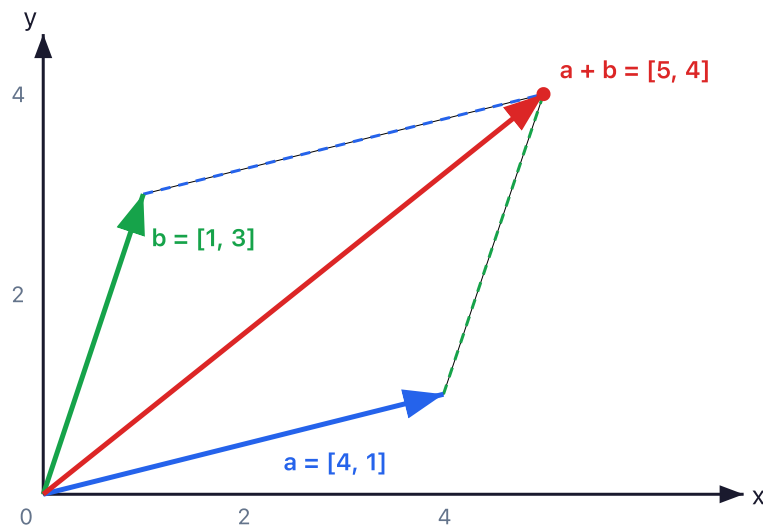
- Vectors are written in **bold** lower case (**a**, **b**) or with an arrow; components use subscripts, so **a** = $[a_1, a_2, \dots, a_n]$.
- The **dimension** n is the number of components. Two vectors can only be added if they have the same dimension.
- The **zero vector** **0** = $[0, 0, \dots, 0]$ is the identity for addition: **a** + **0** = **a**.
- The **magnitude** (length) of **a** is $|\mathbf{a}| = \sqrt{a_1^2 + \dots + a_n^2}$.
- A vector may be drawn as an *arrow* from the origin to the point with those coordinates — convenient in 2 or 3 dimensions, but the algebra works for any n .

Vector Addition

Vector addition is *component-wise*: **a** + **b** = $[a_1 + b_1, a_2 + b_2, \dots, a_n + b_n]$. Geometrically, the sum is the arrow you reach by placing **b**'s tail at **a**'s tip — the diagonal of the parallelogram formed by the two vectors.

Worked example. With **a** = $[4, 1]$ and **b** = $[1, 3]$:

$$\mathbf{a} + \mathbf{b} = [4 + 1, 1 + 3] = [5, 4]$$



The dashed arrows are copies of **a** and **b** shifted to the tip of the other vector. Both routes arrive at the same point $[5, 4]$, which is why addition is *commutative*: $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$.

Scalar–Vector Multiplication

Multiplying a vector by a **scalar** (a single real number) multiplies every component: $k\mathbf{a} = [k \times a_1, k \times a_2, \dots, k \times a_n]$. This stretches the arrow by a factor of k , keeping its direction — or reverses it when k is negative.

Worked example. With $\mathbf{u} = [2, 3, -1]$ and $k = 2.5$:

$$2.5\mathbf{u} = [2.5 \times 2, 2.5 \times 3, 2.5 \times (-1)] = [5.0, 7.5, -2.5]$$

$$-1\mathbf{u} = [-2, -3, 1] \quad // \text{ same length, opposite direction}$$

Addition and scalar multiplication combine, so expressions such as $3\mathbf{a} - 2\mathbf{b}$ are evaluated component-wise: with $\mathbf{a} = [4, 1]$ and $\mathbf{b} = [1, 3]$ this gives $[12 - 2, 3 - 6] = [10, -3]$.

The Dot (Scalar) Product

The **dot product** (also called the *scalar product*) of two vectors of the same dimension multiplies matching components and adds the results: $\mathbf{a} \cdot \mathbf{b} = a_1b_1 + a_2b_2 + \dots + a_nb_n$. The

answer is a single *number*, not a vector.

Worked example (2-D). $\mathbf{a} = [4, 1]$, $\mathbf{b} = [1, 3]$:

$$\mathbf{a} \cdot \mathbf{b} = 4 \times 1 + 1 \times 3 = 4 + 3 = 7$$

Worked example (3-D). $\mathbf{u} = [2, 3, -1]$, $\mathbf{v} = [5, 0, 4]$:

$$\mathbf{u} \cdot \mathbf{v} = 2 \times 5 + 3 \times 0 + (-1) \times 4 = 10 + 0 - 4 = 6$$

What the number *means*: the dot product measures how much the two vectors point the same way, because $\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos\theta$, where θ is the angle between them. Three consequences worth remembering:

- **Perpendicular vectors give zero.** $[3, 4] \cdot [-4, 3] = -12 + 12 = 0$, so those two arrows meet at a right angle. A positive result means the vectors point broadly the same way; a negative result means broadly opposite.
- **A vector dotted with itself gives its squared length.** $\mathbf{a} \cdot \mathbf{a} = 4^2 + 1^2 = 17$, so $|\mathbf{a}| = \sqrt{17} \approx 4.123$.
- **It recovers the angle.** For $\mathbf{a} = [4, 1]$ and $\mathbf{b} = [1, 3]$: $|\mathbf{b}| = \sqrt{10} \approx 3.162$, so $\cos\theta = 7 / (\sqrt{17} \times \sqrt{10}) = 7 / \sqrt{170} \approx 0.537$, giving $\theta \approx 57.5^\circ$.

Convex Combination

A **convex combination** of two vectors $\mathbf{a}$ and $\mathbf{b}$ is $\lambda\mathbf{a} + (1 - \lambda)\mathbf{b}$ where $0 \leq \lambda \leq 1$. Because the two scalars are non-negative and sum to 1, every convex combination lands *on the straight line segment joining* $\mathbf{a}$ and $\mathbf{b}$ — never outside it.

Worked example. Take $\mathbf{p} = [1, 1]$ and $\mathbf{q} = [7, 5]$, and compute $\lambda\mathbf{p} + (1 - \lambda)\mathbf{q}$:

λ	Calculation	Result	Position on the segment
1	$1 \times [1, 1] + 0 \times [7, 5]$	$[1, 1]$	exactly at $\mathbf{p}$
0.75	$[0.75, 0.75] + [1.75, 1.25]$	$[2.5, 2.0]$	a quarter of the way from $\mathbf{p}$ to $\mathbf{q}$

0.5	$[0.5, 0.5] + [3.5, 2.5]$	$[4.0, 3.0]$	the midpoint
0.25	$[0.25, 0.25] + [5.25, 3.75]$	$[5.5, 4.0]$	three quarters of the way to q
0	$0 \times [1, 1] + 1 \times [7, 5]$	$[7, 5]$	exactly at q

Sliding λ from 1 down to 0 therefore sweeps smoothly from **p** to **q**. That is exactly **linear interpolation**, and it is why convex combinations appear all over computing:

- **Animation / tweening** — an object at position **p** in the first key frame and **q** in the last is drawn at $\lambda \mathbf{p} + (1 - \lambda) \mathbf{q}$ for the in-between frames.
- **Colour blending** — treating RGB colours as 3-vectors, $0.3 \times [255, 0, 0] + 0.7 \times [0, 0, 255] = [76.5, 0, 178.5]$, a purple much nearer blue than red. Cross-fades and gradients are the same calculation per pixel.
- **Averaging** — the mean of two vectors is just the case $\lambda = 0.5$, and weighted averages generalise it to more than two.

Where Vectors Are Used

- **Images** — a greyscale image of 1920×1080 pixels can be read as a single vector of 2,073,600 real numbers, so brightening, cross-fading and averaging images become scalar multiplication and vector addition.
- **Graphics** — positions, directions and velocities of objects in 2-D or 3-D are vectors; moving an object is adding a translation vector, scaling it is scalar multiplication, and lighting calculations use the dot product to find how directly a surface faces the light.
- **Sound** — a sampled audio clip is a vector of amplitudes; mixing two clips is vector addition and changing the volume is scalar multiplication.
- **Machine learning and search** — documents, users or items are described by *feature vectors*, and the dot product (or the angle it gives) measures how similar two of them are.

Vector Spaces

A **vector space** is a set of vectors that is **closed** under vector addition and under scalar multiplication: adding any two members gives another member, and multiplying any member

by any real scalar gives another member. $\mathbb{R}^n$ — the set of all n -vectors over $\mathbb{R}$ — is the standard example.

Closure is the point of the definition. Because $[5, 4]$ and $[10, -3]$ are still 2-vectors, you can keep combining vectors indefinitely and never leave $\mathbb{R}^2$. Note that a convex combination stays inside the space too, but only reaches the segment between the two vectors, whereas allowing *any* scalars (a *linear* combination) reaches the whole plane spanned by them.

Exam Tip: When AQA says "vector" in 7517 it means the mathematical object above, so answer with components, not with arrays that grow. For full marks: state the representation you are using (list, function or dictionary), do addition and scalar multiplication *component-wise*, remember the dot product returns a *single number*, and check that the weights in a convex combination are non-negative and add up to 1. A common lost mark is writing the dot product as a vector — and another is forgetting that addition is only defined between vectors of the same dimension.

Big-O Summary Across Structures

The following table lists *typical average-case* complexities. Worst cases differ (e.g. an unbalanced BST degenerates to $O(n)$, a hash table with a dreadful hash collapses to $O(n)$).

Structure	Access by index	Search by value	Insert	Delete	Space
Array (static)	$O(1)$	$O(n)$	$O(n)$ (shift)	$O(n)$ (shift)	$O(n)$
Resizing (dynamic) array	$O(1)$	$O(n)$	$O(1)$ amortised (append)	$O(n)$ (middle)	$O(n)$
Linked list	$O(n)$	$O(n)$	$O(1)$ at head	$O(1)$ given node	$O(n)$
Stack (array)	–	–	$O(1)$ push	$O(1)$ pop	$O(n)$

Queue (circular)	–	–	$O(1)$ enQueue	$O(1)$ deQueue	$O(n)$
BST (balanced)	–	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$
BST (worst)	–	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Hash table	–	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(n)$
Graph (adj. matrix)	$O(1)$ edge test	–	$O(1)$ edge	$O(1)$ edge	$O(V^2)$
Graph (adj. list)	$O(\text{deg})$ edge test	–	$O(1)$ edge	$O(\text{deg})$ edge	$O(V+E)$

Why appending to a resizing array is $O(1)$ amortised. The array is held in a block of a fixed capacity. While there is spare room, an append writes one element and costs $O(1)$. When the block is full, a larger block (typically double the capacity) is allocated and all n elements are copied across, which costs $O(n)$. Because the capacity doubles, those expensive copies get rarer as the array grows: going from capacity 1 to capacity n copies $1 + 2 + 4 + \dots + n/2 < n$ elements in total. So n appends cost $O(n)$ altogether, which is $O(1)$ each on average — an individual append can still cost $O(n)$ when it triggers a resize.

Summary Checklist

- Distinguish static from dynamic data structures, and abstract data type from implementation.
- Compute the memory address of an element in a 1D, 2D or 3D array using row-major and column-major formulas.
- Design a record (composite) data type and distinguish serial, sequential and random file organisation.
- Implement a circular queue using a fixed array plus front, rear and count variables with modulo arithmetic.
- Trace a sequence of enQueue / deQueue operations including a wrap-around.

- Implement a stack using an array and a top pointer; evaluate a Reverse Polish expression using a stack trace.
- Compare array-based and linked-list-based stack / queue implementations.
- Use correct graph terminology (vertex, edge, directed, weighted, connected, degree, cycle).
- Compare adjacency matrix and adjacency list in terms of space ($O(V^2)$ vs $O(V+E)$) and time trade-offs.
- Insert keys into a BST, then give pre-order, in-order and post-order traversals of the resulting tree.
- Write recursive pseudocode for BST insert, search and in-order traversal.
- Describe a hash function's required properties; explain chaining and linear probing; define load factor and rehashing.
- State typical implementations of the dictionary ADT (hash table, balanced BST) and their complexities.
- Represent a vector as a list, as a function from a finite index set to $\mathbb{R}$, or as a dictionary, and convert between the three.
- Carry out vector addition, scalar–vector multiplication, the dot product and a convex combination, and say what a vector space is.